

how to use a bit vector to represent a dynamic set of distinct elements drawn from the set $\{0, 1, \dots, m - 1\}$ and with no satellite data. Dictionary operations should run in $O(1)$ time.

11.1-3

Suggest how to implement a direct-address table in which the keys of stored elements do not need to be distinct and the elements can have satellite data. All three dictionary operations (INSERT, DELETE, and SEARCH) should run in $O(1)$ time. (Don't forget that DELETE takes as an argument a pointer to an object to be deleted, not a key.)

★ 11.1-4

Suppose that you want to implement a dictionary by using direct addressing on a *huge* array. That is, if the array size is m and the dictionary contains at most n elements at any one time, then $m \gg n$. At the start, the array entries may contain garbage, and initializing the entire array is impractical because of its size. Describe a scheme for implementing a direct-address dictionary on a huge array. Each stored object should use $O(1)$ space; the operations SEARCH, INSERT, and DELETE should take $O(1)$ time each; and initializing the data structure should take $O(1)$ time. (*Hint*: Use an additional array, treated somewhat like a stack whose size is the number of keys actually stored in the dictionary, to help determine whether a given entry in the huge array is valid or not.)

11.2 Hash tables

The downside of direct addressing is apparent: if the universe U is large or infinite, storing a table T of size $|U|$ may be impractical, or even impossible, given the memory available on a typical computer. Furthermore, the set K of keys *actually stored* may be so small relative to U that most of the space allocated for T would be wasted.

When the set K of keys stored in a dictionary is much smaller than the universe U of all possible keys, a hash table requires much less storage than a direct-address table. Specifically, the storage requirement reduces to $\Theta(|K|)$ while maintaining the benefit that searching for an

element in the hash table still requires only $O(1)$ time. The catch is that this bound is for the *average-case time*,¹ whereas for direct addressing it holds for the *worst-case time*.

With direct addressing, an element with key k is stored in slot k , but with hashing, we use a *hash function* h to compute the slot number from the key k , so that the element goes into slot $h(k)$. The hash function h maps the universe U of keys into the slots of a *hash table* $T[0 : m - 1]$:

$$h : U \rightarrow \{0, 1, \dots, m - 1\},$$

where the size m of the hash table is typically much less than $|U|$. We say that an element with key k *hashes* to slot $h(k)$, and we also say that $h(k)$ is the *hash value* of key k . Figure 11.2 illustrates the basic idea. The hash function reduces the range of array indices and hence the size of the array. Instead of a size of $|U|$, the array can have size m . An example of a simple, but not particularly good, hash function is $h(k) = k \bmod m$.

There is one hitch, namely that two keys may hash to the same slot. We call this situation a *collision*. Fortunately, there are effective techniques for resolving the conflict created by collisions.

Of course, the ideal solution is to avoid collisions altogether. We might try to achieve this goal by choosing a suitable hash function h . One idea is to make h appear to be “random,” thus avoiding collisions or at least minimizing their number. The very term “to hash,” evoking images of random mixing and chopping, captures the spirit of this approach. (Of course, a hash function h must be deterministic in that a given input k must always produce the same output $h(k)$.) Because $|U| > m$, however, there must be at least two keys that have the same hash value, and avoiding collisions altogether is impossible. Thus, although a well-designed, “random”-looking hash function can reduce the number of collisions, we still need a method for resolving the collisions that do occur.

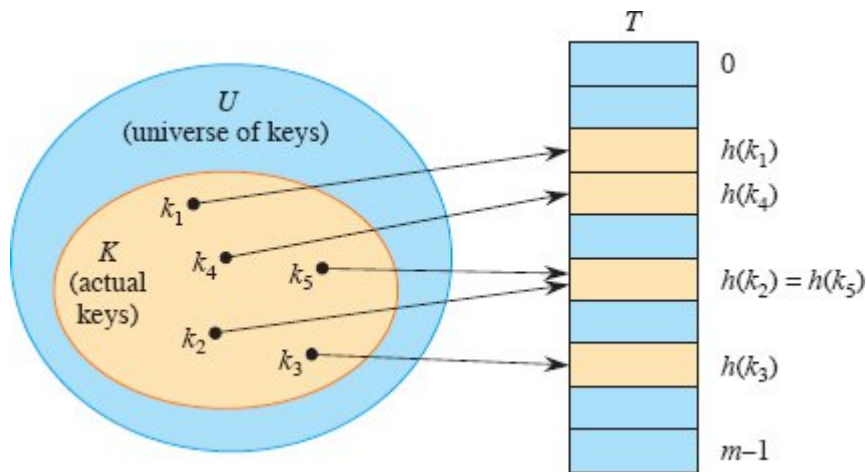


Figure 11.2 Using a hash function h to map keys to hash-table slots. Because keys k_2 and k_5 map to the same slot, they collide.

The remainder of this section first presents a definition of “independent uniform hashing,” which captures the simplest notion of what it means for a hash function to be “random.” It then presents and analyzes the simplest collision resolution technique, called chaining. Section 11.4 introduces an alternative method for resolving collisions, called open addressing.

Independent uniform hashing

An “ideal” hashing function h would have, for each possible input k in the domain U , an output $h(k)$ that is an element randomly and independently chosen uniformly from the range $\{0, 1, \dots, m-1\}$. Once a value $h(k)$ is randomly chosen, each subsequent call to h with the same input k yields the same output $h(k)$.

We call such an ideal hash function an *independent uniform hash function*. Such a function is also often called a *random oracle* [43]. When hash tables are implemented with an independent uniform hash function, we say we are using *independent uniform hashing*.

Independent uniform hashing is an ideal theoretical abstraction, but it is not something that can reasonably be implemented in practice. Nonetheless, we’ll analyze the efficiency of hashing under the

assumption of independent uniform hashing and then present ways of achieving useful practical approximations to this ideal.

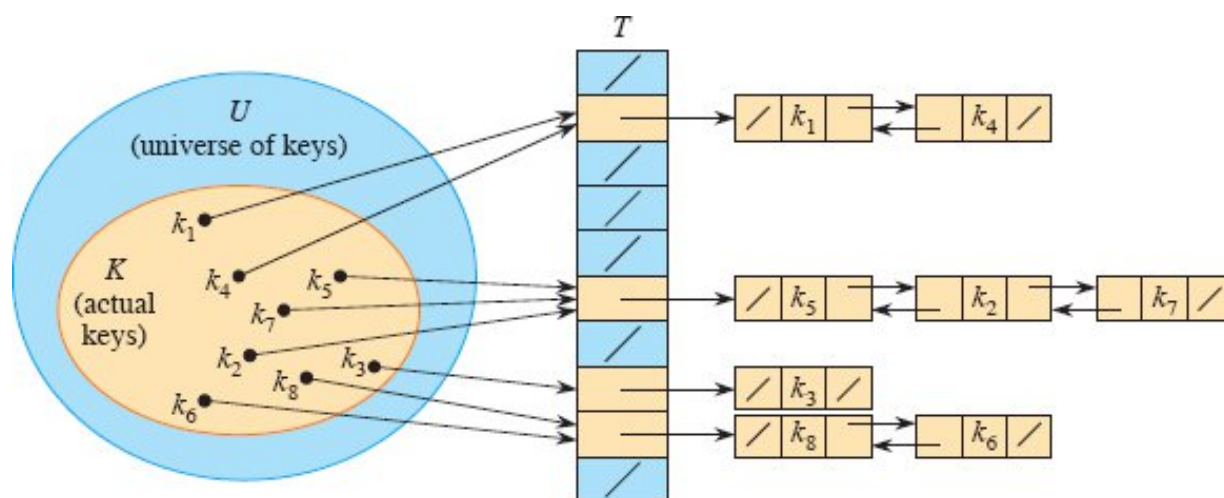


Figure 11.3 Collision resolution by chaining. Each nonempty hash-table slot $T[j]$ points to a linked list of all the keys whose hash value is j . For example, $h(k_1) = h(k_4)$ and $h(k_5) = h(k_2) = h(k_7)$. The list can be either singly or doubly linked. We show it as doubly linked because deletion may be faster that way when the deletion procedure knows which list element (not just which key) is to be deleted.

Collision resolution by chaining

At a high level, you can think of hashing with chaining as a nonrecursive form of divide-and-conquer: the input set of n elements is divided randomly into m subsets, each of approximate size n/m . A hash function determines which subset an element belongs to. Each subset is managed independently as a list.

Figure 11.3 shows the idea behind **chaining**: each nonempty slot points to a linked list, and all the elements that hash to the same slot go into that slot's linked list. Slot j contains a pointer to the head of the list of all stored elements with hash value j . If there are no such elements, then slot j contains NIL.

When collisions are resolved by chaining, the dictionary operations are straightforward to implement. They appear on the next page and use the linked-list procedures from Section 10.2. The worst-case running time for insertion is $O(1)$. The insertion procedure is fast in

part because it assumes that the element x being inserted is not already present in the table. To enforce this assumption, you can search (at additional cost) for an element whose key is $x.key$ before inserting. For searching, the worst-case running time is proportional to the length of the list. (We'll analyze this operation more closely below.) Deletion takes $O(1)$ time if the lists are doubly linked, as in Figure 11.3. (Since CHAINED-HASH-DELETE takes as input an element x and not its key k , no search is needed. If the hash table supports deletion, then its linked lists should be doubly linked in order to delete an item quickly. If the lists were only singly linked, then by Exercise 10.2-1, deletion could take time proportional to the length of the list. With singly linked lists, both deletion and searching would have the same asymptotic running times.)

```
CHAINED-HASH-INSERT( $T, x$ )
1 LIST-PREPEND( $T[h(x.key)], x$ )

CHAINED-HASH-SEARCH( $T, k$ )
1 return LIST-SEARCH( $T[h(k)], k$ )

CHAINED-HASH-DELETE( $T, x$ )
1 LIST-DELETE( $T[h(x.key)], x$ )
```

Analysis of hashing with chaining

How well does hashing with chaining perform? In particular, how long does it take to search for an element with a given key?

Given a hash table T with m slots that stores n elements, we define the **load factor** α for T as n/m , that is, the average number of elements stored in a chain. Our analysis will be in terms of α , which can be less than, equal to, or greater than 1.

The worst-case behavior of hashing with chaining is terrible: all n keys hash to the same slot, creating a list of length n . The worst-case time for searching is thus $\Theta(n)$ plus the time to compute the hash

function—no better than using one linked list for all the elements. We clearly don't use hash tables for their worst-case performance.

The average-case performance of hashing depends on how well the hash function h distributes the set of keys to be stored among the m slots, on the average (meaning with respect to the distribution of keys to be hashed and with respect to the choice of hash function, if this choice is randomized). Section 11.3 discusses these issues, but for now we assume that any given element is equally likely to hash into any of the m slots. That is, the hash function is *uniform*. We further assume that where a given element hashes to is *independent* of where any other elements hash to. In other words, we assume that we are using *independent uniform hashing*.

Because hashes of distinct keys are assumed to be independent, independent uniform hashing is *universal*: the chance that any two distinct keys k_1 and k_2 collide is at most $1/m$. Universality is important in our analysis and also in the specification of universal families of hash functions, which we'll see in Section 11.3.2.

For $j = 0, 1, \dots, m - 1$, denote the length of the list $T[j]$ by n_j , so that

$$n = n_0 + n_1 + \dots + n_{m-1}, \quad (11.1)$$

and the expected value of n_j is $E[n_j] = \alpha = n/m$.

We assume that $O(1)$ time suffices to compute the hash value $h(k)$, so that the time required to search for an element with key k depends linearly on the length $n_{h(k)}$ of the list $T[h(k)]$. Setting aside the $O(1)$ time required to compute the hash function and to access slot $h(k)$, we'll consider the expected number of elements examined by the search algorithm, that is, the number of elements in the list $T[h(k)]$ that the algorithm checks to see whether any have a key equal to k . We consider two cases. In the first, the search is unsuccessful: no element in the table has key k . In the second, the search successfully finds an element with key k .

Theorem 11.1

In a hash table in which collisions are resolved by chaining, an unsuccessful search takes $\Theta(1 + \alpha)$ time on average, under the

assumption of independent uniform hashing.

Proof Under the assumption of independent uniform hashing, any key k not already stored in the table is equally likely to hash to any of the m slots. The expected time to search unsuccessfully for a key k is the expected time to search to the end of list $T[h(k)]$, which has expected length $E[nh(k)] = \alpha$. Thus, the expected number of elements examined in an unsuccessful search is α , and the total time required (including the time for computing $h(k)$) is $\Theta(1 + \alpha)$. ■

The situation for a successful search is slightly different. An unsuccessful search is equally likely to go to any slot of the hash table. A successful search, however, cannot go to an empty slot, since it is for an element that is present in one of the linked lists. We assume that the element searched for is equally likely to be any one of the elements in the table, so the longer the list, the more likely that the search is for one of its elements. Even so, the expected search time still turns out to be $\Theta(1 + \alpha)$.

Theorem 11.2

In a hash table in which collisions are resolved by chaining, a successful search takes $\Theta(1 + \alpha)$ time on average, under the assumption of independent uniform hashing.

Proof We assume that the element being searched for is equally likely to be any of the n elements stored in the table. The number of elements examined during a successful search for an element x is 1 more than the number of elements that appear before x in x 's list. Because new elements are placed at the front of the list, elements before x in the list were all inserted after x was inserted. Let x_i denote the i th element inserted into the table, for $i = 1, 2, \dots, n$, and let $k_i = x_i.\text{key}$.

Our analysis uses indicator random variables extensively. For each slot q in the table and for each pair of distinct keys k_i and k_j , we define the indicator random variable

$X_{ijq} = I \{ \text{the search is for } x_i, h(k_i) = q, \text{ and } h(k_j) = q \}.$

That is, $X_{ijq} = 1$ when keys k_i and k_j collide at slot q and the search is for element x_i . Because $\Pr\{\text{the search is for } x_i\} = 1/n$, $\Pr\{h(k_i) = q\} = 1/m$, $\Pr\{h(k_j) = q\} = 1/m$, and these events are all independent, we have that $\Pr\{X_{ijq} = 1\} = 1/nm^2$. Lemma 5.1 on page 130 gives $E[X_{ijq}] = 1/nm^2$.

Next, we define, for each element x_j , the indicator random variable

$$Y_j = I \{ x_j \text{ appears in a list prior to the element being searched for} \}$$

$$= \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq} ,$$

since at most one of the X_{ijq} equals 1, namely when the element x_i being searched for belongs to the same list as x_j (pointed to by slot q), and $i < j$ (so that x_i appears after x_j in the list).

Our final random variable is Z , which counts how many elements appear in the list prior to the element being searched for:

$$Z = \sum_{j=1}^n Y_j .$$

Because we must count the element being searched for as well as all those preceding it in its list, we wish to compute $E[Z + 1]$. Using linearity of expectation (equation (C.24) on page 1192), we have

$$\begin{aligned}
E[Z + 1] &= E\left[1 + \sum_{j=1}^n Y_j\right] \\
&= 1 + E\left[\sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} X_{ijq}\right] \\
&= 1 + E\left[\sum_{q=0}^{m-1} \sum_{j=1}^n \sum_{i=1}^{j-1} X_{ijq}\right] \\
&= 1 + \sum_{q=0}^{m-1} \sum_{j=1}^n \sum_{i=1}^{j-1} E[X_{ijq}] \quad (\text{by linearity of expectation})
\end{aligned}$$

Thus, the total time required for a successful search (including the time for computing the hash function) is $\Theta(2 + \alpha/2 - \alpha/2n) = \Theta(1 + \alpha)$. ■

What does this analysis mean? If the number of elements in the table is at most proportional to the number of hash-table slots, we have $n = O(m)$ and, consequently, $\alpha = n/m = O(m)/m = O(1)$. Thus, searching takes constant time on average. Since insertion takes $O(1)$ worst-case time and deletion takes $O(1)$ worst-case time when the lists are doubly linked (assuming that the list element to be deleted is known, and not just its key), we can support all dictionary operations in $O(1)$ time on average.

The analysis in the preceding two theorems depends only on two essential properties of independent uniform hashing: uniformity (each key is equally likely to hash to any one of the m slots), and independence (so any two distinct keys collide with probability $1/m$).

Exercises

11.2-1

You use a hash function h to hash n distinct keys into an array T of length m . Assuming independent uniform hashing, what is the expected number of collisions? More precisely, what is the expected cardinality of $\{\{k_1, k_2\} : k_1 \neq k_2 \text{ and } h(k_1) = h(k_2)\}$?

11.2-2

Consider a hash table with 9 slots and the hash function $h(k) = k \bmod 9$. Demonstrate what happens upon inserting the keys 5, 28, 19, 15, 20, 33, 12, 17, 10 with collisions resolved by chaining.

11.2-3

Professor Marley hypothesizes that he can obtain substantial performance gains by modifying the chaining scheme to keep each list in sorted order. How does the professor's modification affect the running time for successful searches, unsuccessful searches, insertions, and deletions?

11.2-4

Suggest how to allocate and deallocate storage for elements within the hash table itself by creating a “free list”: a linked list of all the unused slots. Assume that one slot can store a flag and either one element plus a pointer or two pointers. All dictionary and free-list operations should run in $O(1)$ expected time. Does the free list need to be doubly linked, or does a singly linked free list suffice?

11.2-5

You need to store a set of n keys in a hash table of size m . Show that if the keys are drawn from a universe U with $|U| > (n - 1)m$, then U has a subset of size n consisting of keys that all hash to the same slot, so that the worst-case searching time for hashing with chaining is $\Theta(n)$.

11.2-6

You have stored n keys in a hash table of size m , with collisions resolved by chaining, and you know the length of each chain, including the length L of the longest chain. Describe a procedure that selects a key uniformly at random from among the keys in the hash table and returns it in expected time $O(L \cdot (1 + 1/\alpha))$.

11.3 Hash functions

For hashing to work well, it needs a good hash function. Along with being efficiently computable, what properties does a good hash function